

Abstraction

* It is hiding the actual implementation (without showing implementation)

Ex: Car → Steering (we can't see functionality that how steering connected to wheels) but mechanism is different.
Car → Engine (Internal mechanism is different).

* Abstract methods:

→ We write only declaration to that methods, we cannot write any implementation.

Ex:

```
public void sayHi() {  
    syso("Hi");  
}
```


→ concrete methods.
→ Declaration
→ Implementation.

Syntax: `public abstract void sayHi();`

* We will write method with only body.

* These are written for the purpose, to override by others or inherit by other to get child class to get implementation. We can give implementation to child class only.

* If we want to create an abstract method, abstract keyword must be ~~written~~ used.

* Abstract methods can exist only inside abstract class.

* Abstract class:

* Abstract classes are classes which have ^{both} abstract methods and concrete methods (normal methods) in Java.

* We can mark a class as abstract without an abstract method, we can have only concrete methods also.

* When we inherit the abstract class, definitely we need to implement the unimplemented method in abstract class.

Ex:

package com;

public class Animal {

abstract

public void sound();

abstract method.

[∴ Animals have different types of sounds]

public abstract int noOfLegs();

public void sleeping();

concrete method

syso("sleeping...");

}

public Animal() {

Constructor

}

}

package com;

public class Lion extends Animal {

@Override

public void sound() {

syso("Roar...");

}

public int noOfLegs() {

return 4;

}

*

* We can give implementation to abstract class with overriding the method only (unimplemented method of abstract class).


```
package com;
```

```
public class Test {
```

```
    public static void main (String[] args) {
```

```
        Lion lion = new Lion();
```

```
        lion.sound();
```

```
        lion.sleeping();
```

```
        lion.noOfLegs();
```

```
        Dog dog = new Dog();
```

```
        dog.sound();
```

```
        dog.sleeping();
```

```
        Animal animal = new Animal ();
```

```
    }
```

```
package com;
```

```
public class Test Dog extends Animal {
```

```
    @Override
```

```
    public void sound() {
```

```
        sysout ("Bow Bow...");
```

```
    }
```

```
}
```

Output: Roar...

Sleeping...

4 Bow Bow...

Sleeping...

[∵ We can't create an object to (abstract class), due to that there is an unimplemented method]

*** We can write a constructor for abstract class, but we can't able to create an object to abstract class.

So, there is no use to ~~use~~ write a constructor.

*** Abstract methods cannot be static, because static methods cannot be overridden.

*** Abstract methods cannot be final, because final methods cannot be overridden.

package com;
public class abstract Lion extends Animal {

@Override

public void sound () {

 syso("Roar...")

@Override

public ~~int~~ ~~no~~ legs () {

 return 4;

}
public void run () {

package com;

public class cub extends Lion {

 public void run () {

 syso("Cub is running slowly")

 }

}

package com;

public class Test {

 psum {

 cub. sound ();

 cub. sleeping ();

 }

output: Roar...
 Sleeping...

* We can write abstract keyword to child class also,
to that we have to inherit that child class, ~~also~~
implement the unimplemented methods.

Interfaces

* It looks like a class, but when we create syntax is different.

Syntax: `public interface interfacename {`

`}` *** Interfaces, we cannot create constructors due to this there is no concrete methods i.e., no need to use.

* It allows you to get 100% abstraction.

* It is a combination of abstract methods in Java, till Java 7. only after that they changed interfaces.

*** All methods are by default abstract in interfaces, here abstract keyword is not mandatory.

*** We will not be able to write concrete classes in interfaces. methods

* We can write default & static methods only.

*** In interfaces, all variables are by default public static final. we cannot write private variables, but we need not mention that.

Ex: package com;

`public interface Vehicle {`

`public void start();` → Abstract method by default.

→ `int a = 10;` [must be initialized due to it is a (public static final variable) final variable].

→ `int price = 100000000;` [It is a public variable, JVM will convert default to public]

`}`

*** We cannot create object for for Interfaces.

* When we create child to interface, we have to use implements keyword.

* Inherit:

* class - class - extends.

* interface - child class - implements

* interface - interface - extends.

Ex:

```
package com;
```

```
public interface Vehicle {
```

```
    int price = 100000000;
```

```
    public void start();
```

```
}
```

```
package com;
```

```
public class Car implements Vehicle {
```

```
    @Override
```

```
    public void start() {
```

```
        System.out.println("Car started...");
```

```
    }
```

```
}
```

```
package com;
```

```
public class Test {
```

```
    public static void main (String[] args) {
```

```
        Car car = new Car();
```

```
        car.start();
```

```
        System.out.println(Vehicle.price); // Static Variable.
```

```
        System.out.println(car.price);
```

```
    }
```

```
}
```

Output: Car started ...

100000000.

- * Interfaces enforces some rules, child class should be followed.
→ child class should be implemented that rules enforced by interface.

Ex:

```
package com.interfaces;
```

```
public interface Trait {
```

```
    public void calling();
```

```
    public void data();
```

```
    public void messaging();
```

```
    public void recharge();
```

```
}
```

```
package com.interfaces;
```

```
public class Airtel implements Trait {
```

```
    @Override
```

```
    public void calling() {
```

```
        syso("call at 1p1s");
```

```
}
```

```
    @Override
```

```
    public void data() {
```

```
        syso("2G");
```

```
}
```

```
    @Override
```

```
    public void messaging() {
```

```
        syso("100 sms");
```

```
}
```

```
    @Override
```

```
    public void recharge() {
```

```
        syso("1GB at 100rs");
```

```
}
```

```
package com.interfaces;
```

```
public class Jio implements Trait {
```

```
    @Override
```

```
    public void calling() {
```

```
        syso("unlimited");
```


@Override

```
public class data() {  
    syso("4GB");  
}
```

@Override

```
public class messaging() {  
    syso("Unlimited messaging");  
}
```

@Override

```
public class recharge() {  
    syso("150/- per month");  
}
```

```
package com.interfaces;
```

```
public class Test {
```

```
    Psvm {
```

```
        Airtel airtel = new Airtel();
```

```
        airtel.calling();
```

```
        airtel.data();
```

```
        airtel.messaging();
```

```
        airtel.recharge();
```

```
        Jio jio = new Jio();
```

```
        jio.calling();
```

```
        jio.data();
```

```
        jio.messaging();
```

```
        jio.recharge();
```

```
    }
```

```
}
```

Output: Calling at 11/11/18

2G

100 SMS

1GB at 100 rs

Unlimited

4GB

Unlimited messaging

150/- per month.

* Object class is not a parent to
 interfaces.

* Multiple Inheritance is supported in Interfaces.

* Ex: package com;

```
public interface A {
    public void m1();
    public void m2();
}
```

[∴ * Interface do not have implementation.]

* Child of class gives that implementation.

* We cannot override same methods, so, it happens only once in child class.]

* default & static methods can give Implementation in Java-8.

due to this abstr interfaces lost 100% abstraction.

package com;

```
public interface B {
```

```
    public void m1();
```

```
    public void m3();
```

```
    [not an access modifier] default void m4() {
        syso("Hi...");
    }
```

```
package com;
```

```
public class C implements A, B {
```

```
    @Override
```

```
    public void m1() {
```

```
        syso("Hi I am m1");
    }
```


@Override

```
public void m2() {
```

```
    sysout("Hi I am m2");
```

```
}
```

@Override

```
public void m3() {
```

```
    sysout("Hi I am m3");
```

```
}
```

```
package com;
```

```
public class Test {
```

```
    private
```

```
        Psum {
```

```
        c obj = new c();
```

```
        c.m1();
```

```
        c.m2();
```

```
        c.m3();
```

```
}
```

Output: Hi I am M1

Hi I am M2

Hi I am M3

Ex 1 package com.interfaces;

```
public interface Sim {  
    int a = 10;  
    public long IMEI();  
    public long aadhaarNum();  
    public long number();  
}
```

}

package com.interfaces;

public class N1 implements Sim {

@Override

public long IMEI() {

System.out.println("IMEI: 1234567");

}

@Override

public long aadhaarNum() {

return 4523692540;

}

@Override

public long ~~aadhaar~~ number() {

return 8540698232;

}

package com. interfaces;

public class Uninos implements sim {

@ Override int c = 30;

public long IMEI () {

sys return IMEI;

1245679

}

@ Override

public long aadhaarNum () {

return aadhaarNum;

}

4730 0568 9872

@ Override

public long number () {

return 8540096823;

}

package com. interfaces;

public class Test {

psum {

VI Vi = new VI ();

long number = Vi. number ();

long IMEI = Vi. IMEI ();

long aadhaar = Vi. ^{aadhaarNum} ~~number~~ ();

Num =

sys (number)

sys (IMEI)

sys (aadhaarNum);

Unind unind = new Unind ()

long number2 = unind. number ()

sys ("Unind number:" + number2);

* Parent Variable holding a child class object - reference

Sim sim = new VI (); → we can call like this.

syso(sim.number()); → output: 8540695232

It works with class also)

Sim sim = new Unindl ();

syso(sim.number()); → output: 8540096823;

* Parent class reference can hold child class objects.

Sim sim = new Sim (); [we can't create (error) object to interface]

syso (Sim.a); → output: 10

* When we hold it in the parent class reference variable it will refer only to the parent variables & methods.

* We can access methods in Sim interface with parent reference variable, we cannot access own implementations given in child classes.

* Overridden values can be accessed only with parent referencing variable, even if we modify values of parent class. ⇒ syso(sim.a); output: 10

*

~~***~~ ~~*~~ We can change VISIBILITY access modifiers, but we cannot ~~give~~ change to weaker access modifiers.

(public > protected > private > default)

Ex: package com;

```
public class Employee {
```

```
    public void m1() {  
        syso ("Hi m1");  
    }
```

```
    public protected void m2() {  
        syso ("Hi m2");  
    }
```

```
    public private void m3() {  
        syso ("Hi m3");  
    }
```

```
    public default void m4() {  
        syso ("Hi m4");  
    }
```

```
}
```

[* We cannot assign weaker access modifier - ex]

[We cannot reduce the access, but we can give higher access]

package com;

```
public class EmployeeChild extends Employee {
```

@Override

```
    public void m1() {  
        syso ("From Child m1");  
    }
```

@Override

```
    protected void m2() {  
        syso ("From child m2");  
    }
```

```
}
```

@Override

```
    ✓ protected private void m4() {  
        syso ("From child m4");  
    }
```

✗ private

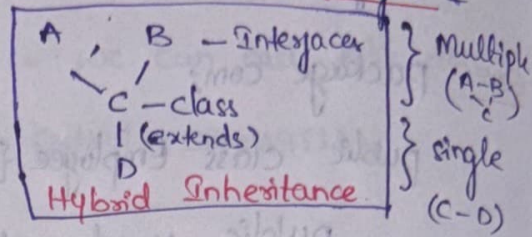
✓ public

```
}
```


Hybrid Inheritance.

* It is a combination of ~~many~~ ^{two} inheritances, i.e, single & multiple inheritances.

Ex: package com;
public ~~class~~ ^{interface} Vehicle {
public void start();
syso("starting")



package com;
public ~~class~~ ^{interface} Car {

public void carName();
public

package com;

public class FuelCar implements Vehicle, Car {

@Override,

public void start() {

syso("Vehicle started");

}

@ Override

```
public void carName() {  
    syso("Car");  
}
```

package com;

```
public class Bmw extends FuelCar {
```

@ Override

```
public void CarName() {  
    syso("BMW Car");  
}  
public void price() {
```

```
    syso(100000000);  
}
```

package com;

```
public class Test {
```

```
    psvm {
```

```
        Bmw bmw = new Bmw();
```

```
        bmw.start(); // we can access, because it  
                      // coming from grandparent (ultimate  
                      // parent)
```

Output:

Vehicle started.